

22

Lecture 22

**Introduction to Nand2Tetris and Jack
Language Basics**

by - Brijesh Shukla
C07: Operating Systems

Today's Agenda

- The Big Picture
- The Hack Computer Platform
- The Software Hierarchy
- Building from Scratch: The "Why"
- Jack Language: Syntax & Structure
- Logic & Constructs in Jack



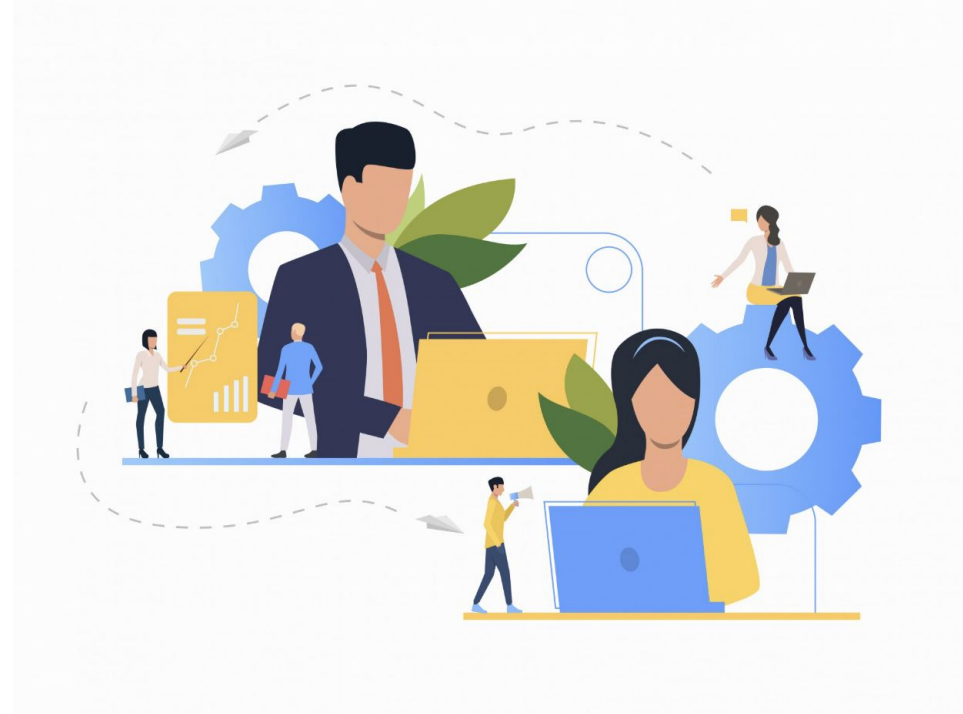
The Journey So Far

Processes and scheduling: How the conductor manages the orchestra.

Memory management and paging: How the library organizes its books.

I/O systems and device management: How the manager interacts with staff and external resources.

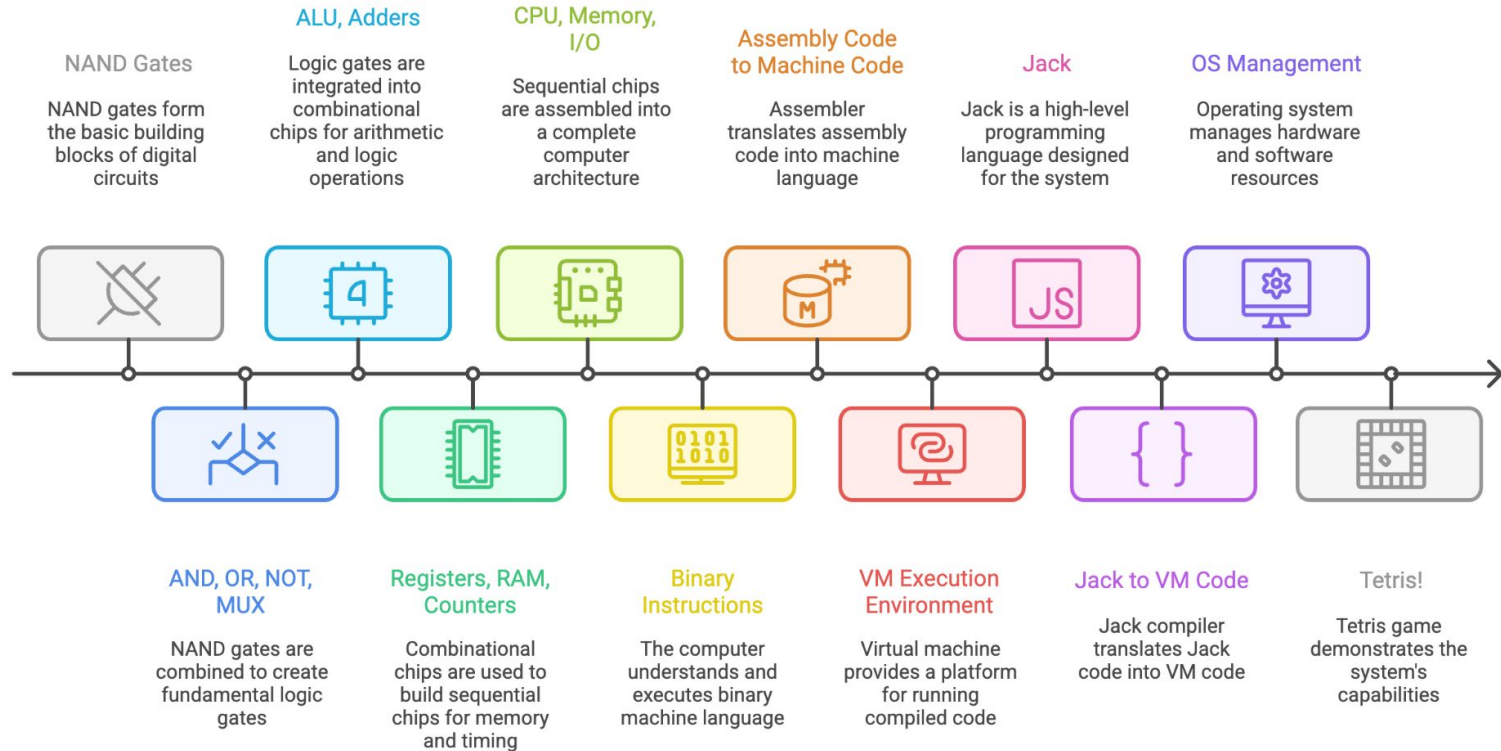
How do these components actually get built from the ground up?



- Have you ever wondered what happens between writing `int main()` and the CPU executing instructions?

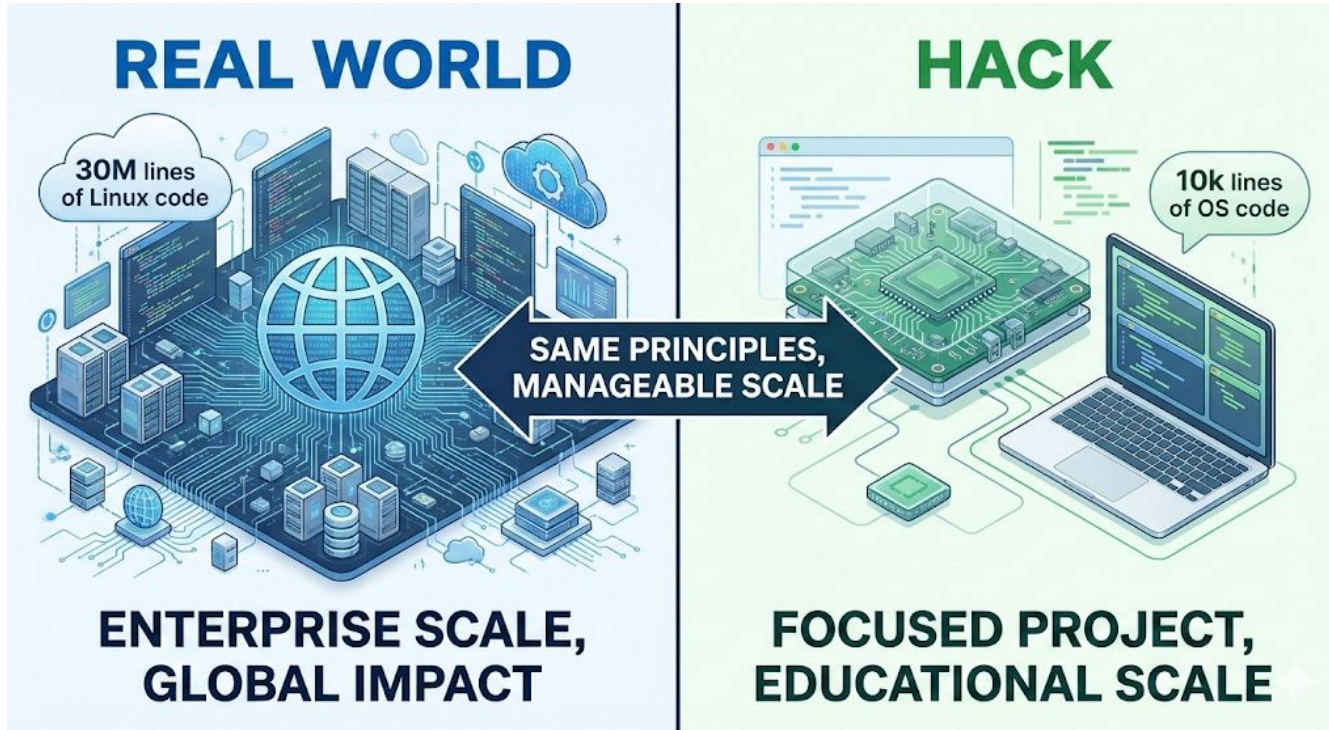


The Grand Vision



Why Build from Scratch?

Deep intuition beats abstract knowledge.



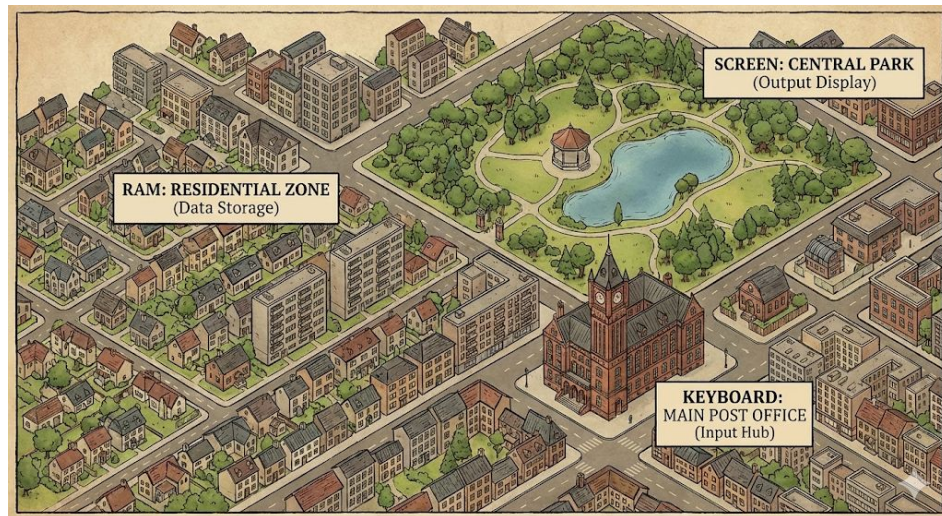
The Hack Computer Platform

16-bit Harvard Architecture. Minimalist design.

CPU Registers : "A" and "D" Registers.

The ALU (Arithmetic Logic Unit) : The calculator of the system.

Memory Mapping (The Map): **32K words**.
RAM (**0-16K**), Screen (**16K-24K**),
Keyboard (**24K**).

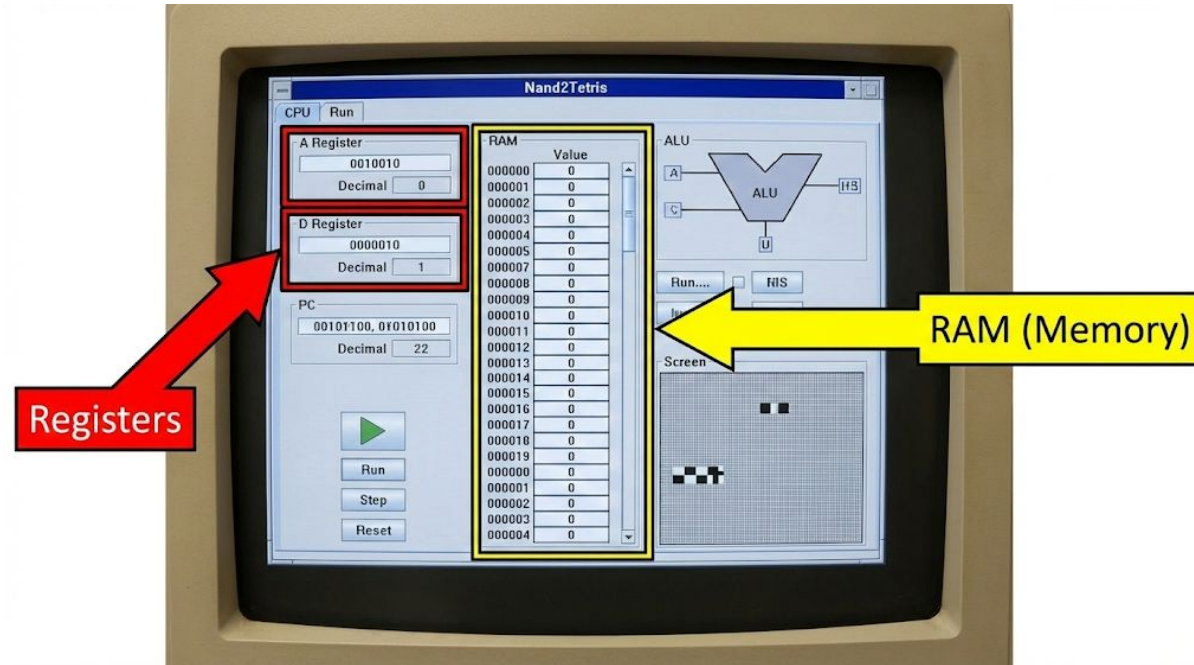


The Hack Computer Platform

Screen Memory: 256 rows x 512 columns. 1 bit = 1 pixel.

Keyboard I/O: Reading address 24576.

The CPU Emulator



English (Jack)

Simplified Code (VM)

Symbols (Assembly)

1s and 0s (Machine).



Machine Language (Layer 0)

The "Ground Floor" of software.

Binary Instructions: 16-bit strings like
00000000000000010 (A-instruction) or
1110110000010000 (C-instruction).

Characteristics: Directly executed by
hardware; extremely tedious for
humans; zero abstraction.



The "DNA" of the computer

Assembly Language (Layer 1)

Symbolic Representation: `@sum` instead of `@16`.

The Assembler's Job:

- Translate symbols to addresses.
- Convert mnemonics (`D=M`) to binary.
- Resolve labels and output `.hack` files.

Assembly Language (Source)	Binary (Machine Code)
	10111000 00000101
MOV AX, 05h	10111000 00000101
MOV BX, 0Ah	10111011 00001010
ADD AX, BX	➡ 00000001 11000011
JMP START	11101001 11111100

Like using a contact name (e.g., "Mom") in your phone instead of memorizing her 10-digit phone number.

Virtual Machine (Layer 4)

Intermediate Abstraction: A stack-based model

VM Commands: `push constant 5,`
`add, pop local 0.`

Why VM?

Platform Independence: Write once, run on any hardware with a VM translator.

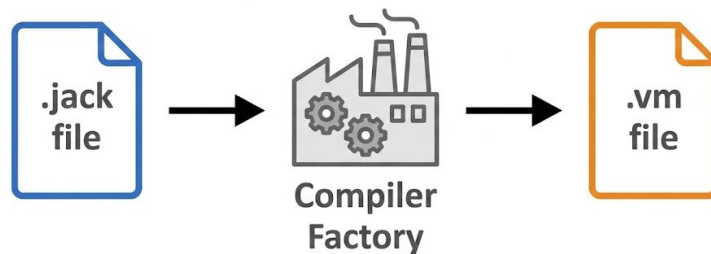
Simplifies Compiler: The Jack compiler only needs to know how to talk to the VM.



The Jack Compiler (Layer 5)

Compilation Pipeline:

- **Tokenizer:** Breaking text into "words" (keywords, symbols).
- **Parser:** Understanding the "grammar" (syntax tree).
- **Code Generator:** Emitting the VM instructions.



Jack Language (Layer 6)

High-level, Object-Oriented.

Features: Classes, methods, fields, and static variables.

Role: The bridge between human logic and the VM translator.

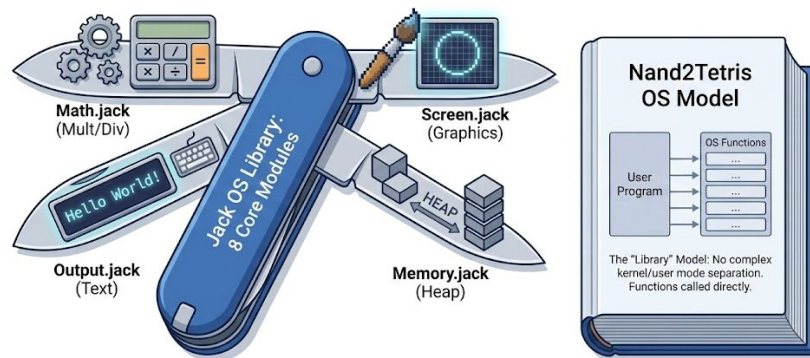
```
class Main {  
    function int addPair(int x, int y) {  
        var int result;  
        let result = x + y;  
        return result;  
    }  
}
```


The Operating System (Layer 6)

Jack OS Library: 8 Core Modules.

`Math.jack` (Mult/Div), `Output.jack` (Text),
`Screen.jack` (Graphics), `Memory.jack`
(Heap), `String.jack`, `Keyboard.jack`,
`Sys.jack`

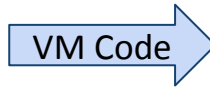
The "Library" Model: In Nand2Tetris, the OS is a set of functions called directly by your program—no complex kernel/user mode separation.



The Jack OS Swiss Army Knife: Pre-made tools you don't want to build yourself.

Translation Example

let x = 5 + 3;

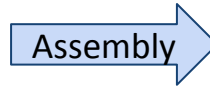


push constant 5

push constant 3

add

pop local 0



@5

0000000000000101

D=A

1110110000010000

@SP

0000000000000000

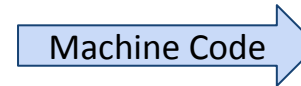
A=M

// ... (binary)

M=D

@SP

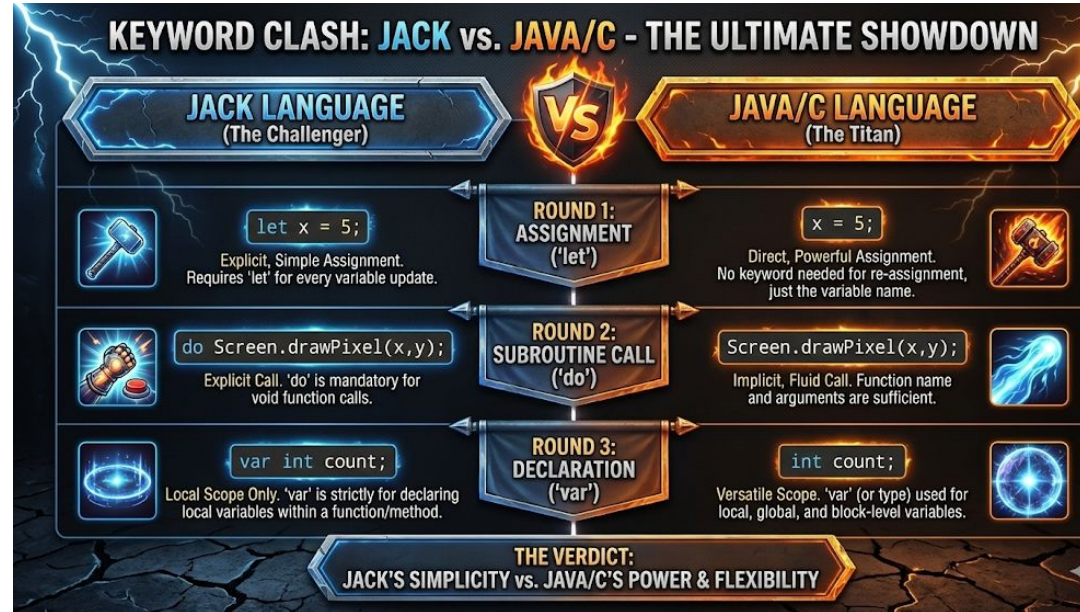
M=M+1



Jack vs Java/C

Similarities: Statically typed, OO structure, curly braces.

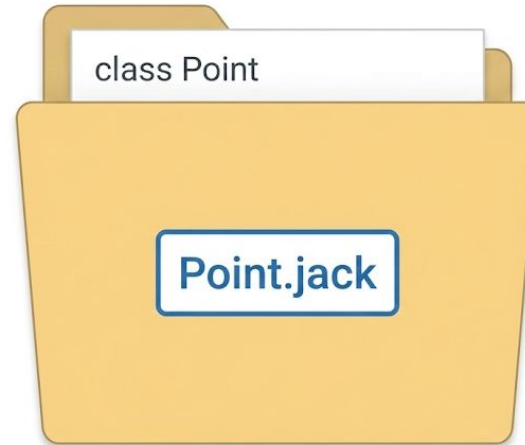
Differences: No inheritance, no interfaces, very simple memory management, no garbage collection (in the basic version).



Jack File Structure

Rule: One class per file. Filename must match ClassName.

Structure: Class declaration ->
Fields/Statics -> Constructor ->
Subroutines.



Primitive Data Types

`int`: 16-bit signed (-32768 to 32767).

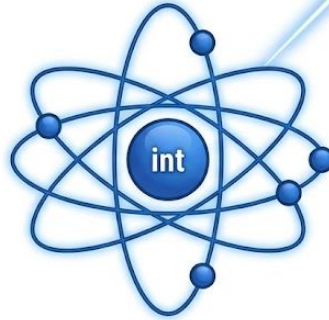
`char`: ASCII character.

`boolean`: true or false.

Arrays & Strings: Handled as objects using `Array.new(size)` and `String.new(length)`.

THE SIMPLEST BUILDING BLOCKS

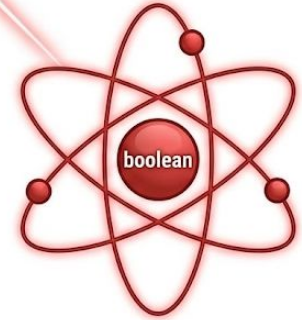
JACK PRIMITIVE DATA TYPES: THE ATOMS



Integer: Whole Numbers



Character: Single Symbols



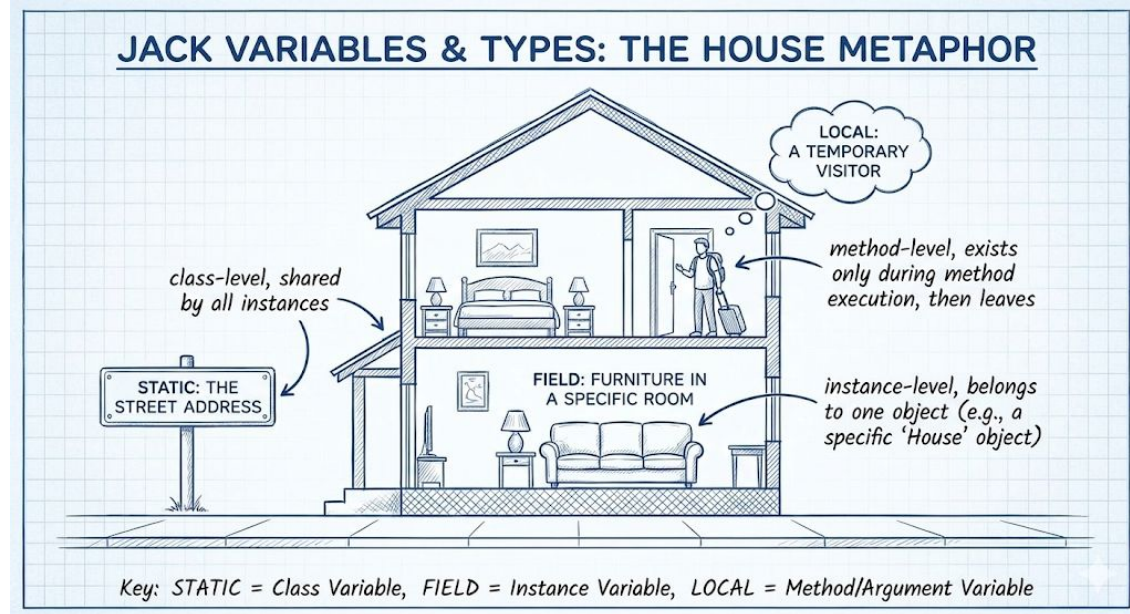
Boolean: True/False

Variable Types & Scope

Field: One per object instance (State).

Static: Shared by all instances of a class.

Local: Exists only during subroutine execution (Stack).



Memory Management (Jack Style)

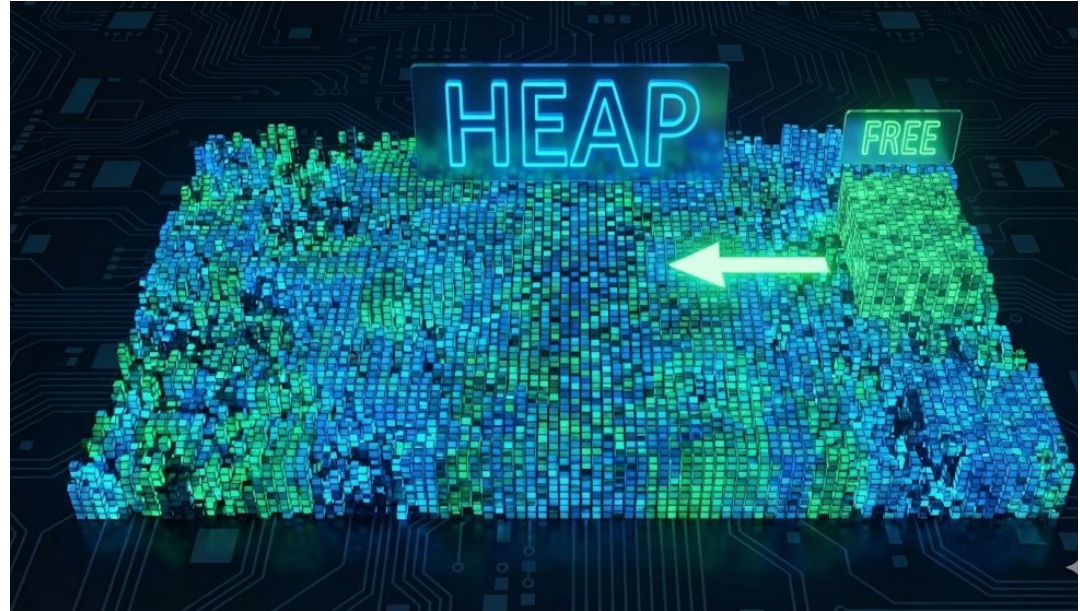
Heap Management: Handled by
`Memory.jack`.

```
let arr = Array.new(10); ->
```

Allocates block.

```
do arr.dispose(); ->
```

Freeing
the block manually.

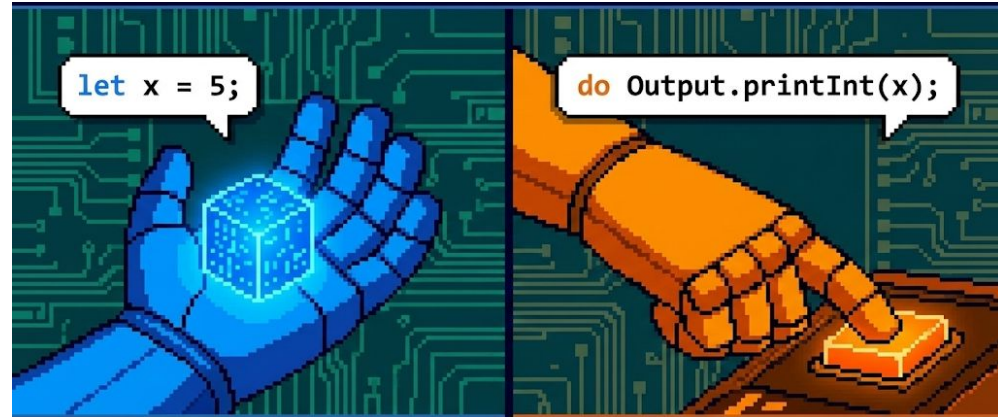


Basic Statement: let and do

let: Used for assignments. `let x = 5;`

do: Used for subroutine calls when you ignore the return value.

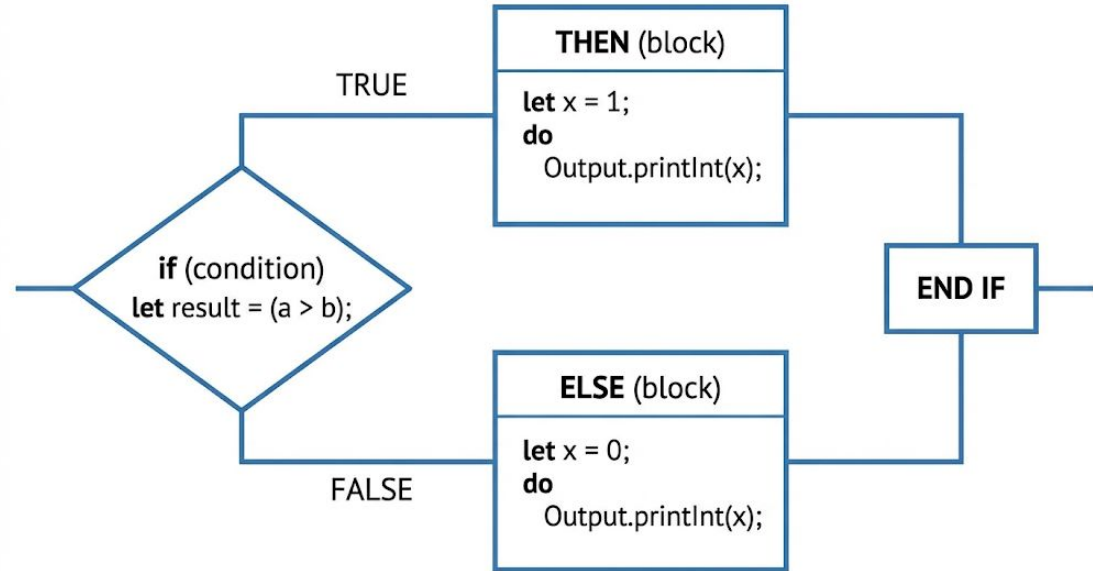
`do Screen.drawPixel(x, y);.`



Control Flow: If-Else

Syntax: `if (condition) { ...
} else { ... }.`

Conditions must be boolean expressions.



Control Flow: While

The only loop construct in Jack!

Syntax: `while (condition) {`
`... }.`

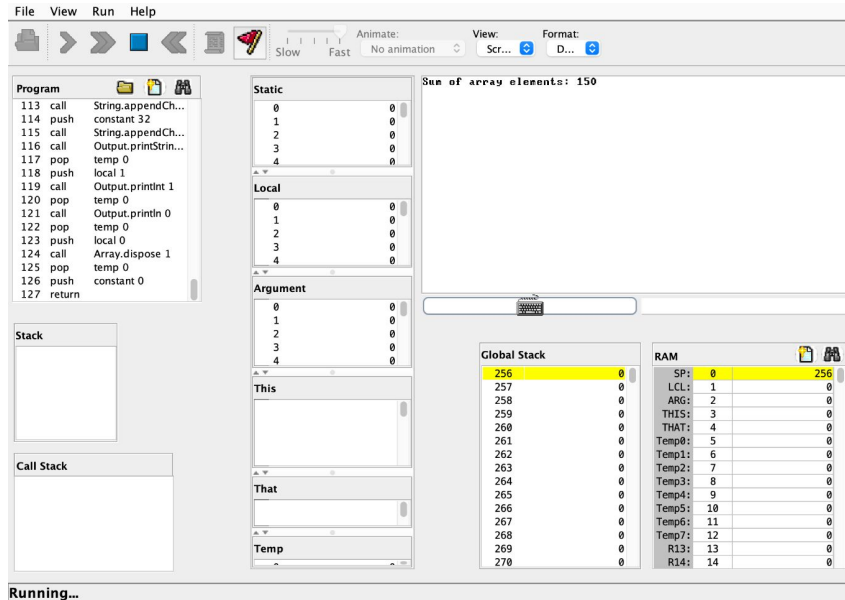
```
let sum = 0;
let i = 0;

while (i < length) {
    let sum = sum + arr[i];
    let i = i + 1;
}

// Display the result
do Output.printString("Sum of array elements: ");
do Output.printInt(sum);
do Output.println();
```

Jack Software Suite

<https://drive.google.com/file/d/1IkIR8Pwg3PY49QgXpUJOOkUUVht-TKIET/view>



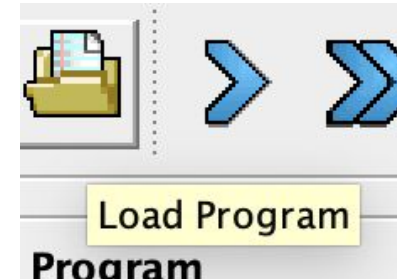
The screenshot displays the Jack Software Suite IDE interface. The top menu bar includes File, View, Run, and Help. Below the menu is a toolbar with icons for file operations and execution. The main workspace is divided into several panels:

- Program:** A list of assembly instructions with line numbers 113 to 127. The instructions include `String.appendCh...`, `push constant 32`, `String.appendCh...`, `Output.printStrin...`, `pop temp 0`, `push local 1`, `Output.printint 1`, `pop temp 0`, `Output.printin 0`, `pop temp 0`, `push local 0`, `Array.dispose 1`, `pop temp 0`, `push constant 0`, and `return`.
- Static:** A table showing static variables with indices 0 to 4, all containing the value 0.
- Local:** A table showing local variables with indices 0 to 4, all containing the value 0.
- Argument:** A table showing argument variables with indices 0 to 4, all containing the value 0.
- This:** A table showing the 'this' pointer, currently empty.
- That:** A table showing the 'that' pointer, currently empty.
- Temp:** A table showing temporary variables, currently empty.
- Global Stack:** A table showing memory addresses from 256 to 270, all containing the value 0.
- RAM:** A table showing memory addresses from 0 to 14, all containing the value 0.
- Stack:** A table showing the current stack, currently empty.
- Call Stack:** A table showing the call stack, currently empty.

The status bar at the bottom indicates the program is **Running...**

Steps

- Create your source files with the `.jack` extension
- Translate your high-level Jack code into Virtual Machine language.
 - Run the `Jack Compiler` script (batch/shell) located in your tools folder.
 - `~/Desktop/nand2tetris/tools/JackCompiler.sh`
- Open the `VMEulator`
 - `~/Desktop/nand2tetris/tools/VMEulator.sh`
- Select "Load Program" (or Load Folder) and choose the directory containing your new `.vm` files.
- Set animation speed to "Fast" or "No Animation" for complex games.
- If logic errors occur, revise `.jack` files and re-compile before reloading in the VM Emulator.



Some Jack Programs

Sum of array Elements

```
let arr[0] = 10;  
let arr[1] = 20;  
let arr[2] = 30;  
let arr[3] = 40;  
let arr[4] = 50;
```

Output
Sum of array elements: 150

Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.